

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1**

o0o



BÀI TẬP LỚN NHẬP MÔN TRÍ TUỆ NHÂN TẠO

**Tên đề tài: Sử dụng thuật toán tìm kiếm A* để tìm ra
đường đi ngắn nhất từ điểm xuất phát đến điểm đích
trong một lưới không gian 2 chiều có vật cản.**

LỚP : N09

Số thứ tự nhóm: 04

Nguyễn Văn Bằng

MSSV: B23DCCN067

Tô Quang Trung

MSSV: B23DCCN863

Nguyễn Trung Tường

MSSV: B23DCCN909

Phạm Quang Toàn

MSSV: B23DCCN835

Giảng viên hướng dẫn: Ths. Hoài Thư

HÀ NỘI, 05/2026

LỜI CẢM ƠN

Trong suốt quá trình thực hiện bài tập lớn “Sử dụng thuật toán A* để tìm ra đường đi ngắn nhất trong lưới không gian hai chiều có vật cản”, nhóm chúng em đã nhận được rất nhiều sự giúp đỡ, động viên và hỗ trợ quý báu từ nhiều phía. Đây không chỉ là một bài tập học thuật, mà còn là hành trình rèn luyện tư duy, sự kiên trì và tinh thần làm việc nhóm, để lại cho chúng em nhiều bài học ý nghĩa.

Trước hết, nhóm xin bày tỏ lòng biết ơn sâu sắc tới giảng viên hướng dẫn – ThS. Hoài Thư. Cô không chỉ truyền đạt kiến thức nền tảng một cách rõ ràng, logic mà còn luôn tận tình định hướng, góp ý chi tiết và kịp thời trong suốt quá trình nhóm thực hiện đề tài. Những nhận xét của cô không chỉ giúp nhóm hoàn thiện bài làm mà còn giúp chúng em hiểu sâu hơn về bản chất của thuật toán, cách tư duy trong nghiên cứu và cách tiếp cận một vấn đề một cách khoa học. Sự nghiêm túc và tâm huyết của cô chính là động lực để chúng em cố gắng hoàn thiện bài tập lớn với chất lượng tốt nhất.

Bên cạnh đó, chúng em xin cảm ơn những người bạn, những người đồng đội trong nhóm đã cùng nhau hợp tác, chia sẻ kiến thức, hỗ trợ lẫn nhau và không ngừng cố gắng. Trong quá trình làm việc, chắc chắn không tránh khỏi những lúc bất đồng quan điểm hay gặp khó khăn trong việc triển khai ý tưởng, nhưng chính tinh thần trách nhiệm và sự đoàn kết đã giúp nhóm vượt qua tất cả. Mỗi thành viên đều đã nỗ lực hết mình, đóng góp trí tuệ và thời gian để hoàn thành đề tài một cách trọn vẹn nhất.

Mặc dù đã nỗ lực hết sức, bài tập lớn chắc chắn vẫn còn những hạn chế nhất định. Nhóm rất mong nhận được sự thông cảm và những ý kiến đóng góp quý báu từ giảng viên để có thể tiếp tục hoàn thiện hơn trong tương lai.

Một lần nữa, nhóm xin chân thành cảm ơn tất cả những sự hỗ trợ, đồng hành và tin tưởng đã giúp chúng em hoàn thành bài tập lớn này. Đây sẽ là nền tảng quan trọng để chúng em tiếp tục học tập và phát triển trong lĩnh vực Trí tuệ nhân tạo cũng như trong con đường học thuật phía trước.

MỤC LỤC

LỜI CẢM ƠN

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI..... 1

1.1 Đặt vấn đề..... 1

1.2 Mục tiêu và phạm vi đề tài..... 1

1.3 Định hướng giải pháp..... 2

1.4 Bố cục bài tập lớn 3

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 5

2.1 Giới thiệu bài toán tìm đường đi ngắn nhất từ điểm xuất phát đến điểm đích trong một lưới không gian 2 chiều có vật cản 5

2.1.1 Khái niệm bài toán 5

2.1.2 Ứng dụng 5

2.1.3 Một số khó khăn khi giải quyết bài toán 6

2.2 Thuật toán A^* 6

2.2.1 Giới thiệu về thuật toán A^* 6

2.2.2 Ý tưởng của thuật toán A^* 6

2.2.3 Mã giả của thuật toán 6

2.2.4 Hàm heuristic 8

2.2.5 Tính chất thuật toán A^* 9

2.2.6 Ưu và nhược điểm của thuật toán A^* 10

2.3 Ứng dụng thuật toán A^* cho bài toán tìm đường đi ngắn nhất từ điểm xuất phát đến điểm đích trong một lưới không gian 2 chiều có vật cản 10

2.3.1 Thiết lập bài toán trong chương trình 10

2.3.2 Xác định trạng thái và ràng buộc khi di chuyển trong lưới..... 10

2.3.3 Tổ chức dữ liệu và quá trình tìm đường trong chương trình 11

2.3.4 Trực quan hóa quá trình tìm kiếm	11
CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ.....	13
3.1 Mục tiêu thực nghiệm	13
3.1.1 Mục tiêu tổng quát	13
3.1.2 Mục tiêu cụ thể	13
3.2 Triển khai hệ thống.....	14
3.2.1 Thiết kế chức năng	14
3.2.2 Giao diện người dùng	14
3.2.3 Công cụ và thư viện.....	15
3.3 Dữ liệu và bộ test.....	16
3.3.1 Dữ liệu đầu vào.....	16
3.3.2 Các kịch bản kiểm thử	16
3.3.3 Mục đích bộ test.....	16
3.4 Phương pháp đánh giá.....	17
3.4.1 Các tiêu chí đánh giá	17
3.4.2 Quy trình tiến hành thu thập dữ liệu.....	17
3.4.3 Phương pháp phân tích và trình bày kết quả	17
3.5 Kết quả thực nghiệm	18
3.5.1 Kết quả kiểm thử tự động (Test Runner)	18
3.5.2 Kết quả so sánh Heuristic.....	22
3.6 Phân tích và kết luận	24
3.6.1 Giải thích các kết quả	24
3.6.2 Kết luận	25
CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	27
4.1 Kết luận.....	27

4.2 Đóng góp chính	27
4.2.1 Kiến trúc generator cho tìm kiếm thời gian thực.....	27
4.2.2 Khung so sánh đa heuristic tích hợp.....	28
4.2.3 Cơ chế kiểm soát ràng buộc di chuyển linh hoạt.....	28
4.3 So sánh với các công trình liên quan.....	28
4.4 Bài học kinh nghiệm.....	28
4.5 Hướng phát triển.....	29
4.5.1 Các công việc cần thiết để hoàn thiện sản phẩm hiện tại	29
4.5.2 Hướng phát triển mở rộng: Multi-Heuristic A*	29

DANH MỤC HÌNH VẼ

Hình 2.1	Ví dụ chạy bằng thuật toán A*	8
Hình 2.2	Ví dụ hàm heuristic chấp nhận được	9
Hình 3.1	Giao diện trong trường hợp vật cản ngẫu nhiên	15
Hình 3.2	Giao diện trong trường hợp mê cung	15
Hình 3.3	Kết quả Test Scenario 1	19
Hình 3.4	Kết quả Test Scenario 2	20
Hình 3.5	Kết quả Test Scenario 3	21
Hình 3.6	Kết quả Test Scenario 4	22
Hình 3.7	Bảng so sánh T1	22
Hình 3.8	Bảng so sánh T2	23
Hình 3.9	Bảng so sánh T3	23
Hình 3.10	Bảng so sánh T4	23
Hình 3.11	Bảng so sánh T5	23

DANH MỤC BẢNG BIỂU

Bảng 1.1	So sánh các phương pháp tìm đường tiêu biểu	2
----------	---	---

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
A*	Thuật toán tìm đường A-Star
API	Giao diện lập trình ứng dụng (Application Programming Interface)
CPU	Bộ vi xử lý trung tâm (Central Processing Unit)
GUI	Giao diện người dùng đồ họa (Graphical User Interface)
IDE	Môi trường phát triển tích hợp (Integrated Development Environment)
JSON	Định dạng trao đổi dữ liệu (JavaScript Object Notation)
UI	Giao diện người dùng (User Interface)
UX	Trải nghiệm người dùng (User Experience)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Bài toán tìm đường đi ngắn nhất trên lưới hai chiều phát sinh từ nhiều ứng dụng thực tiễn. Trong robot di động tự hành, khả năng xác định hành trình tối ưu từ vị trí xuất phát đến điểm đích trên bản đồ có chướng ngại vật là yêu cầu nền tảng cho điều khiển và tránh va chạm. Các hệ thống trò chơi điện tử chiến thuật và nhập vai trực tuyến cũng yêu cầu hàng nghìn thực thể ảo đồng thời tính toán đường đi với chi phí xử lý thấp. Lĩnh vực logistics kho bãi và hệ thống giao thông thông minh đối mặt với bài toán tương tự trên không gian lưới được số hóa.

Thuật toán A* do Hart, Nilsson và Raphael công bố trên IEEE Transactions năm 1968 đặt nền tảng lý thuyết cho bài toán này. Nguyên lý cốt lõi của A* là hàm đánh giá $f(n) = g(n) + h(n)$, trong đó $g(n)$ là chi phí thực tế từ điểm xuất phát đến đỉnh n , $h(n)$ là chi phí ước lượng từ n đến đích. Chứng minh tính tối ưu của A* khi $h(n)$ là hàm chấp nhận được tạo ra bước đột phá so với các thuật toán tìm kiếm mù như Dijkstra hay BFS. Hiệu năng của A* trong không gian trạng thái lớn phụ thuộc vào ba yếu tố: lựa chọn hàm heuristic, khả năng di chuyển theo đường chéo, và chiến lược tìm kiếm đơn hướng so với song hướng.

Khi bài toán tìm đường được giải quyết hiệu quả, các hệ thống ứng dụng thu được ba lợi ích chính. Robot giảm thời gian di chuyển và tiết kiệm năng lượng tiêu thụ trong quá trình hoạt động. Trò chơi điện tử tăng số lượng thực thể có thể xử lý đồng thời trên cùng nền tảng phần cứng. Hệ thống thời gian thực đáp ứng được các ràng buộc về độ trễ tính toán. Các giải pháp này còn có thể mở rộng sang lĩnh vực lập quỹ đạo cho cánh tay robot trong không gian làm việc nhiều chiều, hoặc tối ưu hóa lộ trình vận chuyển trong kho tự động hóa. Phần tiếp theo sẽ khảo sát các nghiên cứu hiện có để xác định mục tiêu cụ thể cho đề tài.

1.2 Mục tiêu và phạm vi đề tài

Các nghiên cứu cải tiến A* trên lưới hai chiều tập trung vào ba hướng chính. Hướng thứ nhất là điều chỉnh hàm heuristic với bốn biến thể phổ biến: Manhattan ($|dx| + |dy|$), Euclidean ($\sqrt{dx^2 + dy^2}$), Octile ($\max(dx, dy) + (\sqrt{2} - 1) \times \min(dx, dy)$), Chebyshev ($\max(dx, dy)$). Trường hợp đặc biệt Dijkstra ($h = 0$) biến A* thành thuật toán tìm kiếm không có thông tin heuristic. Hướng thứ hai là cho phép di chuyển góc bất kỳ, tiêu biểu là Theta* do Nash và cộng sự giới thiệu tại AAAI 2007, loại bỏ ràng buộc 8 hướng của lưới truyền thống. Hướng thứ ba là tăng tốc tìm kiếm với Jump Point Search (JPS) của Harabor và Grastien tại AAAI 2011, bỏ qua các đỉnh trung gian không cần thiết trên lưới đồng nhất.

Bảng 1.1: So sánh các phương pháp tìm đường tiêu biểu

Phương pháp	Độ phức tạp thời gian	Chất lượng đường đi	Khả năng tùy chỉnh
A* Manhattan	$O(b^d)$ trung bình	Tối ưu 8 hướng	Cao (4 heuristic)
A* Euclidean	$O(b^d)$ trung bình	Tối ưu góc bất kỳ	Cao (4 heuristic)
Theta*	$O(b^d)$ cao hơn	Tối ưu góc bất kỳ	Trung bình
JPS	$O(n)$ trên lưới đồng nhất	Tối ưu 8 hướng	Thấp (không heuristic)

Phân tích bảng 1.1 cho thấy mỗi phương pháp chỉ tối ưu cho một tập con điều kiện đầu vào cụ thể. A* Manhattan đơn giản và hiệu quả nhưng chỉ tạo đường đi theo 8 hướng. A* Euclidean tạo đường đi tự nhiên hơn nhưng tốn thời gian tính toán căn bậc hai. Theta* tối ưu về độ dài đường đi nhưng chi phí kiểm tra điểm cao. JPS nhanh nhất trên lưới đồng nhất nhưng không hỗ trợ heuristic tùy ý.

Hạn chế lớn nhất của các hệ thống hiện tại là thiếu một khung làm việc tích hợp cho phép so sánh trực quan các biến thể. Người dùng không thể dễ dàng thay đổi heuristic, chế độ đa hướng, hay chiến lược tìm kiếm song hướng trên cùng một nền tảng. Hầu hết các công bố chỉ cung cấp kết quả mô phỏng trên dữ liệu tổng hợp mà không đi kèm công cụ trực quan hóa quá trình mở rộng đỉnh theo thời gian thực.

Trên cơ sở các hạn chế đã chỉ ra, mục tiêu của bài tập lớn này là phát triển một hệ thống phần mềm trực quan hóa và so sánh các biến thể A* trên lưới hai chiều. Hệ thống cần đáp ứng bốn yêu cầu chức năng chính. Cài đặt đầy đủ A* với năm hàm heuristic (Euclidean, Manhattan, Octile, Chebyshev, Dijkstra). Hỗ trợ tùy chọn di chuyển đường chéo và kiểm soát cắt góc theo nguyên tắc không đi xuyên qua góc chướng ngại vật. Cung cấp cơ chế điều chỉnh tốc độ hiển thị. Cho phép lưu và tải cấu hình bản đồ dưới định dạng JSON để tái lập thực nghiệm.

Điểm khác biệt của hệ thống so với các công bố hiện có nằm ở những tính năng mang tính ứng dụng cao. Đầu tiên là hệ thống trực quan hóa toàn bộ quá trình tìm kiếm với mã màu phân biệt ba trạng thái của đỉnh: open (màu xanh lá), closed (màu xanh ngọc), path (màu vàng). Thứ hai là hệ thống thống kê chi tiết các chỉ số như thời gian xử lý (ms), số nút đã duyệt (visited nodes), và số phép tính (operations). Cuối cùng là tính năng Save Map và Load Map, cho phép trích xuất và tái tạo chính xác các không gian lưới phức tạp (Bẫy Heuristic, Mê cung) dưới định dạng JSON. Các nội dung chi tiết về cài đặt sẽ được trình bày trong Chương 3.

1.3 Định hướng giải pháp

Bài tập lớn này đề xuất định hướng giải pháp dựa trên ba quyết định kiến trúc chính. Thứ nhất, sử dụng cơ chế lưu trữ sự kiện (event logging) kết hợp với phát lại có điều khiển, trong đó mỗi bước mở rộng đỉnh được ghi nhận thành cặp (cell_type,

position) vào mảng frames. Thứ hai, tổ chức các tham số heuristic và cấu hình di chuyển (allow_diagonal, dont_cross_corners, diagonal_cost_one) thành các biến điều khiển độc lập, cho phép so sánh công bằng giữa các cấu hình. Thứ ba, xây dựng giao diện trực quan với thư viện Tkinter, trong đó mỗi ô lưới được biểu diễn bằng một đối tượng hình chữ nhật độc lập để thay đổi màu sắc thời gian thực.

Giải pháp cụ thể được tổ chức trong một lớp App duy nhất quản lý toàn bộ dữ liệu và điều khiển. Ma trận grid_data kích thước ROWS×COLUMNS lưu trạng thái các ô (0: đường đi, 1: tường). Cấu trúc dữ liệu tìm kiếm sử dụng dictionary g_scores để lưu chi phí đường đi đến từng ô, set closed_set để đánh dấu ô đã mở rộng, và hàng đợi ưu tiên open_set (heapq) để quản lý các ô đang chờ xét. Hàm _calculate_heuristic đóng gói năm phương pháp heuristic: Euclidean, Manhattan, Octile, Chebyshev, và Dijkstra ($h = 0$).

Hàm tìm kiếm chính _run_astar_algorithm được triển khai theo lưu đồ tuần tự truyền thống nhưng có ghi nhận sự kiện. Mỗi lần một ô được thêm vào open set, hàm ghi ('open', pos) vào mảng frames. Mỗi lần một ô được lấy ra khỏi open set để mở rộng, hàm ghi ('closed', pos) vào mảng frames. Hàm _get_neighbors xử lý ba cấp độ kiểm soát di chuyển: Allow Diagonal (bật/tắt di chuyển chéo), Don't Cross Corners (kiểm tra hai ô kề cạnh trước khi cho phép đi chéo), Diagonal Cost = 1 (gán chi phí chéo bằng 1.0 thay vì $\sqrt{2}$). Quá trình phát lại sử dụng phương thức after() của Tkinter để tạo hiệu ứng hoạt hình tuần tự với độ trễ tùy chỉnh.

Đóng góp chính của bài tập lớn là ba kết quả có thể đo lường và tái tạo. Thứ nhất, một hệ thống phần mềm hoàn chỉnh cho phép người dùng tương tác xây dựng bản đồ bằng chuột (thêm/xóa tường, kéo thả điểm đầu/cuối), lưu trữ và chia sẻ bản đồ bằng JSON (Save/Load Map), thay đổi cấu hình đa dạng (năm heuristic, ba chế độ di chuyển, tốc độ), và quan sát quá trình tìm kiếm với mã màu trực quan. Thứ hai, chức năng Compare All thực hiện lần lượt năm heuristic trên cùng một bản đồ, thu thập bốn chỉ số (Path Cost, Visited Nodes, Execute Time, Operations) và hiển thị kết quả dạng bảng so sánh. Thứ ba, bộ kiểm thử tự động với cấu hình lưu trong file test_cases.json, đảm bảo tính đúng đắn của thuật toán dưới mọi kịch bản môi trường phức tạp.

1.4 Bố cục bài tập lớn

Phần còn lại của báo cáo được tổ chức thành ba chương tiếp theo. Chương 2 trình bày cơ sở lý thuyết của bài toán tìm đường trên đồ thị, bao gồm định nghĩa không gian trạng thái, hàm chi phí, tính chất chấp nhận được của heuristic, và phân tích độ phức tạp thời gian, không gian của A*. Chương 3 giới thiệu toàn bộ quá trình thực nghiệm từ thiết kế kiến trúc phần mềm (cấu trúc lớp App, các biến điều

khuyến, cơ chế ghi nhận sự kiện) đến triển khai các hàm heuristic, hàm lấy lân cận với ba cấp độ kiểm soát di chuyển, hàm tìm kiếm chính, giao diện người dùng với cơ chế kéo thả, điều khiển tốc độ, quản lý lưu/tải bản đồ, cùng chức năng Compare All và bộ kiểm thử tự động. Chương 4 tổng kết các kết quả đạt được, phân tích những hạn chế còn tồn tại, và đề xuất ba hướng phát triển trong tương lai bao gồm tích hợp tìm kiếm song hướng (bidirectional A*), hỗ trợ xuất bản đồ ra file hình ảnh, và tối ưu hiệu năng bằng cách lưu trữ danh sách neighbor tĩnh.

Tổng quan

Chương 1 đã đặt bài toán tìm đường ngắn nhất trên lưới hai chiều và chỉ ra sự thiếu vắng một hệ thống trực quan hóa tích hợp nhiều biến thể A*. Nội dung chính của chương bao gồm: phần 1.1 đặt vấn đề và phân tích bối cảnh thực tiễn phát sinh bài toán; phần 1.2 tổng quan các nghiên cứu liên quan và xác định mục tiêu, phạm vi đề tài; phần 1.3 đề xuất định hướng giải pháp công nghệ; phần 1.4 mô tả cấu trúc toàn bộ báo cáo. Chương tiếp theo sẽ trình bày chi tiết cơ sở lý thuyết về hàm heuristic, tính chất tối ưu của A*, và các công trình nghiên cứu liên quan.

Kết chương

Chương này đã xác định chính xác bài toán tìm đường ngắn nhất trên lưới hai chiều và vị trí của thuật toán A* trong bối cảnh các nghiên cứu hiện đại. Việc phân tích bốn phương pháp (A* Manhattan, A* Euclidean, Theta*, JPS) chỉ ra rằng chưa có giải pháp nào tích hợp đầy đủ bốn tính năng: linh hoạt về heuristic, kiểm soát di chuyển linh hoạt, trực quan hóa thời gian thực, và giao diện tương tác đồ họa. Định hướng giải pháp dựa trên cơ chế lưu trữ sự kiện và phát lại có điều khiển, sử dụng hàng đợi ưu tiên heapq và thư viện Tkinter, được đề xuất để lấp đầy khoảng trống này. Ba đóng góp chính của bài tập lớn là hệ thống trực quan hóa hoàn chỉnh tích hợp tính năng Save/Load Map, chức năng Compare All so sánh năm heuristic, và bộ kiểm thử tự động với dữ liệu linh hoạt. Chương tiếp theo sẽ trình bày chi tiết cơ sở lý thuyết về hàm heuristic, tính chất tối ưu của A*, và các công trình nghiên cứu liên quan.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Tổng quan chương

Trong chương 1, nhóm đã giới thiệu khái quát về đề tài. Trong chương 2, nhóm sẽ trình bày cơ sở lý thuyết là cơ sở cho việc xây dựng và triển khai chương trình. Đầu tiên, nhóm sẽ giới thiệu bài toán tìm đường đi ngắn nhất từ điểm xuất phát đến điểm đích trong lưới không gian hai chiều có vật cản. Đây là bài toán phổ biến trong các lĩnh vực như trí tuệ nhân tạo, robot và trò chơi điện tử. Tiếp theo, nhóm giới thiệu thuật toán A^* , một thuật toán tìm kiếm có thông tin, sử dụng chi phí thực tế và giá trị heuristic để định hướng tìm kiếm. Cuối cùng, chương sẽ trình bày cách ứng dụng thuật toán A^* để giải quyết bài toán tìm đường trong lưới hai chiều có vật cản, làm cơ sở cho chương sau.

2.1 Giới thiệu bài toán tìm đường đi ngắn nhất từ điểm xuất phát đến điểm đích trong một lưới không gian 2 chiều có vật cản

2.1.1 Khái niệm bài toán

Bài toán tìm đường đi ngắn nhất (Shortest Path Problem) là bài toán xác định tuyến đường tối ưu từ một điểm xuất phát đến một điểm đích sao cho tổng chi phí di chuyển là nhỏ nhất. Chi phí này có thể được đo qua nhiều tiêu chí như khoảng cách, thời gian, năng lượng tiêu hao hoặc số bước di chuyển. Trong mô hình lưới không gian 2 chiều có vật cản, môi trường được chia thành các ô vuông đều nhau tạo thành ma trận gồm hàng và cột. Mỗi ô đại diện cho một vị trí trong không gian. Các ô được chia thành: ô trống, ô vật cản, ô bắt đầu, ô đích. Mục tiêu của hệ thống là tìm ra chuỗi các bước di chuyển từ ô bắt đầu tới ô đích thông qua các ô trống đồng thời tránh va chạm với các ô vật cản và đảm bảo quãng đường tìm được là quãng đường ngắn nhất.

2.1.2 Ứng dụng

Bài toán tìm đường đi trong lưới 2 chiều có vật cản được ứng dụng rộng rãi trong nhiều lĩnh vực như:

- Robot tự hành: Robot thường phải di chuyển trong không gian rộng như nhà máy, kho hàng hoặc nhà hàng, vì vậy nó cần có thuật toán để quét môi trường làm đầu vào và xử lý để tìm ra tuyến đường tối ưu và tránh bị va chạm, hỏng hóc nhất có thể.
- Hệ thống dẫn đường: Các hệ thống bản đồ chỉ dẫn khi được cung cấp địa điểm tới thì cũng cần thuật toán để đánh giá, đưa ra tuyến đường nhanh nhất từ vị trí hiện thời tới đích, tránh những khu vực hay tắc nghẽn, đang bị cấm đường,

...

- Trò chơi điện tử: Các tựa game moba như Liên minh huyền thoại, Dota2 cần cơ chế dẫn đường thông minh khi người chơi chỉ định nhân vật tiến tới 1 vị trí trên bản đồ, đảm bảo tuyến đường là ngắn nhất và không va vào địa hình trong trò chơi..

2.1.3 Một số khó khăn khi giải quyết bài toán

Trong quá trình nghiên cứu các giải pháp cho bài toán, người nghiên cứu gặp phải nhiều khó khăn. Khi kích thước của lưới lớn, số trạng thái nhiều hơn làm cho thời gian tính toán tăng lên, gây sức ép lên bộ nhớ có thể xảy ra tràn bộ nhớ. Môi trường có thể thay đổi liên tục đòi hỏi giải thuật phải có tính đáp ứng cao, cho ra kết quả với hầu hết mọi trường hợp. Bên cạnh đó, việc tối ưu tốc độ tính toán và vấn đề lưu trữ cũng là một thách thức lớn đối với người nghiên cứu.

2.2 Thuật toán A*

2.2.1 Giới thiệu về thuật toán A*

Nhóm thuật tìm kiếm có thông tin là các thuật toán sử dụng thêm thông tin phụ từ bài toán để định hướng tìm kiếm. Các thuật toán này sử dụng một giá trị $f(n)$ để đánh giá độ tiềm năng của nút n từ đó chọn ra nút tốt nhất để mở rộng trước. Thuật toán A* là một thuật toán tìm kiếm thuộc nhóm thuật toán trên, rất nổi tiếng trong khoa học máy tính và trí tuệ nhân tạo. A* được đánh giá cao vì thuật toán này có thể tìm được đường đi tối ưu nhất, tốc độ tìm kiếm nhanh và được áp dụng rộng rãi trong nhiều lĩnh vực tiêu biểu như robot và game.

2.2.2 Ý tưởng của thuật toán A*

A* là thuật toán thuộc nhóm tìm kiếm có thông tin nên nó cũng sẽ đánh giá độ tốt của một nút n thông qua $f(n)$. A* sẽ chọn nút n có giá trị $f(n)$ tối ưu nhất để mở rộng trước với:

$$f(n) = g(n) + h(n)$$

Trong đó:

- $g(n)$: chi phí từ điểm bắt đầu đến nút n
- $h(n)$: ước lượng chi phí từ n đến đích

2.2.3 Mã giả của thuật toán

Cách thức hoạt động của thuật toán A* được thể hiện như sau:

Algorithm 1: A* Search

Input: Q, S, G, P, c, h

Output: Trạng thái đích

Khởi tạo: $O \leftarrow \{S\}, P \leftarrow \emptyset$;

while $O \neq \emptyset$ **do**

 Lấy nút n có $f(n)$ nhỏ nhất khỏi O ;

if $n \in G$ **then**

return đường đi tới n ;

for mọi $m \in P(n)$ **do**

$g(m) = g(n) + c(n, m)$;

$f(m) = g(m) + h(m)$;

 thêm m vào O ;

$P[m] = n$;

return Không tìm được đường đi;

Trong quá trình thực hiện, nếu thuật toán gặp nút lặp thì trong trường hợp nút lặp có chi phí tốt hơn, nút lặp này sẽ được đưa lại vào danh sách nếu nó đã được phát triển rồi hoặc được thay thế khi nó vẫn đang còn trong danh sách.

Khi thuật toán chạy ra kết quả, có thể tiến hành truy vết đường đi thông qua cách sau:

Algorithm 2: Truy vết đường đi

Input: G, S, P

Output: Đường đi từ S đến G

Khởi tạo: $tmp \leftarrow G, path \leftarrow []$;

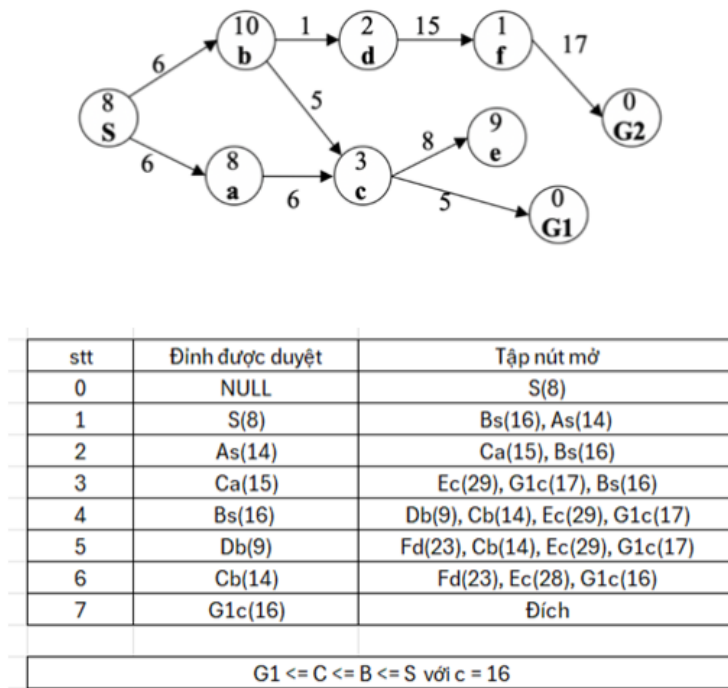
while $tmp \neq S$ **do**

$path.push_back(tmp)$;

$tmp \leftarrow P[tmp]$;

$path.push_back(S)$;

return $path.reverse()$;



Hình 2.1: Ví dụ chạy bằng thuật toán A*

2.2.4 Hàm heuristic

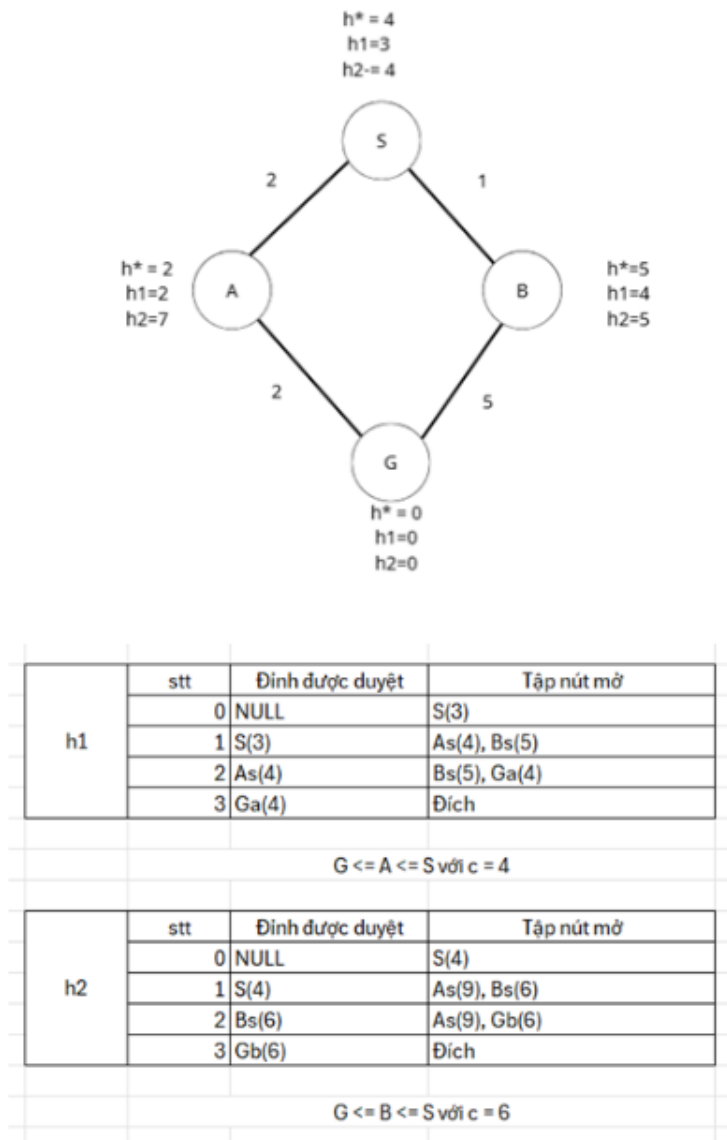
Hàm heuristic là một thành phần quan trọng trong thuật toán A* có ảnh hưởng lớn tới kết quả của thuật toán tìm kiếm. Một bài toán có thể có nhiều hàm heuristic khác nhau. Một số hàm heuristic phổ biến:

Khoảng cách Euclidean: $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

Khoảng cách Manhattan: $h(n) = |x_1 - x_2| + |y_1 - y_2|$

Khoảng cách Chebyshev: $h(n) = \max(|x_1 - x_2|, |y_1 - y_2|)$

Hàm heuristic chấp nhận được là một khái niệm quan trọng để đánh giá tính tối ưu của thuật toán A*. Khái niệm này được định nghĩa như sau: Với mọi nút n , nếu $h(n) \leq h^*(n)$, trong đó $h^*(n)$ là chi phí thực tế tối ưu đi từ n tới đích, thì $h(n)$ được gọi là hàm heuristic chấp nhận được (admissible heuristic). Khi đó, thuật toán A* sử dụng hàm $h(n)$ sẽ đảm bảo tìm được đường đi tối ưu.



Hình 2.2: Ví dụ hàm heuristic chấp nhận được

Hàm heuristic trội là khái niệm dùng để so sánh độ tối ưu giữa các hàm heuristic chấp nhận được. Với hai hàm $h_1(n)$ và $h_2(n)$ đều là hàm heuristic chấp nhận được, nếu với mọi nút n có $h_1(n) \geq h_2(n)$ thì h_1 được coi là trội hơn h_2 .

2.2.5 Tính chất thuật toán A*

Thuật toán A* có tính đầy đủ vì nếu tồn tại đường đi từ nút bắt đầu đến nút kết thúc thì thuật toán sẽ tìm ra lời giải. Điều này có thể sai nếu không gian tìm kiếm là hữu hạn hoặc có vô số nút có chi phí nhỏ hơn lời giải.

Thuật toán A* có độ phức tạp về mặt thời gian là với b là số nhánh trung bình mỗi nút và m là độ sâu của lời giải. Thời gian chạy có thể được cải thiện nếu hàm heuristic đủ tốt để A* xác định được nút gần đích hơn và tránh chạy lan man hơn.

Thuật toán A* có độ phức tạp về mặt không gian là với b là số nhánh trung bình

mỗi nút và m là độ sâu của lời giải. Không gian lưu trữ có thể được cải thiện nếu hàm heuristic đủ tốt.

Thuật toán A^* có tính tối ưu, nghĩa là sẽ tìm ra đường đi ngắn nhất nếu như hàm heuristic được sử dụng là hàm heuristic chấp nhận được.

2.2.6 Ưu và nhược điểm của thuật toán A^*

Thuật toán A^* có nhiều ưu điểm. Đầu tiên thuật toán này không những có thể tìm được đường đi từ điểm xuất phát tới đích mà còn có thể tìm được đường đi tối ưu nếu hàm heuristic được sử dụng là hàm heuristic chấp nhận được. Không những thế, A^* rất linh hoạt vì có thể hoạt động tốt trong nhiều môi trường tiêu biểu như lưới hai chiều, bản đồ giao thông, đồ thị, không gian ba chiều. Thuật toán A^* cũng là một thuật toán dễ cài đặt do chủ yếu sử dụng các cấu trúc dữ liệu như hàng đợi ưu tiên, mảng, danh sách. Thuật toán này cũng có nhiều biến thể mở rộng ví dụ như IDA^* , Weighted A^* , cho phép sử dụng kinh hoạt trong các bài toán khác nhau.

Bên cạnh đó, A^* cũng tồn tại một số nhược điểm. Thuật toán này tiêu tốn tài nguyên bộ nhớ, nhược điểm này càng trở nên rõ ràng hơn khi không gian tìm kiếm lớn làm cho số lượng các trạng thái cần lưu trữ tăng lên. A^* phụ thuộc nhiều vào hàm heuristic, nếu hàm heuristic không tốt có thể làm giảm hiệu năng, độ chính xác và tính tối ưu của thuật toán.

2.3 Ứng dụng thuật toán A^* cho bài toán tìm đường đi ngắn nhất từ điểm xuất phát đến điểm đích trong một lưới không gian 2 chiều có vật cản

2.3.1 Thiết lập bài toán trong chương trình

Trong chương trình, không gian tìm kiếm được biểu diễn dưới dạng một lưới hai chiều kích thước cố định. Mỗi ô trong lưới tương ứng với một trạng thái, được phân loại thành ô trống hoặc ô vật cản. Hai vị trí đặc biệt là điểm bắt đầu và điểm đích được xác định trước hoặc có thể thay đổi thông qua thao tác của người dùng.

2.3.2 Xác định trạng thái và ràng buộc khi di chuyển trong lưới

Từ một ô hiện tại, chương trình xác định các trạng thái kế tiếp thông qua việc xét các ô lân cận. Tùy theo cấu hình, việc di chuyển có thể bao gồm:

- Bốn hướng cơ bản (trên, dưới, trái, phải)
- Bốn hướng cơ bản và bốn hướng chéo (nếu được cho phép)

Ngoài ra, chương trình còn xử lý các ràng buộc thực tế như:

- Không cho phép đi xuyên qua góc tường
- Kiểm soát chi phí di chuyển theo từng hướng

Những yếu tố trên góp phần giúp mô hình bài toán trở nên sát với môi trường thực tế hơn, hữu ích trong các ứng dụng mô phỏng hoặc ứng dụng cho robot

2.3.3 Tổ chức dữ liệu và quá trình tìm đường trong chương trình

Để triển khai thuật toán cần sử dụng các cấu trúc dữ liệu phù hợp. Cấu trúc dữ liệu hàng đợi ưu tiên lưu danh sách các nút mở kèm giá trị $f(n)$ tương ứng, mục đích là để tối ưu hiệu năng vì đầu hàng đợi luôn là nút có trạng thái tốt nhất. Cấu trúc dữ liệu set để đánh dấu các nút đã duyệt vì set cho phép kiểm tra nút đang xét đã được duyệt chưa với tốc độ cao. Ngoài ra cấu trúc dữ liệu map cũng được sử dụng để lưu và truy xuất chi phí đường đi, quan hệ cha con giữa các nút một cách nhanh chóng.

Quá trình tìm đường được thực hiện theo cơ chế lặp. Ban đầu, điểm bắt đầu được đưa vào danh sách mở với chi phí bằng 0. Trong mỗi bước lặp, chương trình chọn ra trạng thái có giá trị đánh giá nhỏ nhất để mở rộng. Nếu trạng thái này trùng với điểm đích, quá trình tìm kiếm kết thúc. Nếu chưa đạt đích, trạng thái hiện tại sẽ được chuyển sang danh sách đóng, và các trạng thái lân cận của nó sẽ được xét. Với mỗi trạng thái lân cận, chương trình tính toán chi phí mới dựa trên chi phí đã có và chi phí di chuyển. Nếu đường đi mới tốt hơn, thông tin về chi phí và quan hệ cha sẽ được cập nhật, đồng thời trạng thái này được đưa vào danh sách mở. Quá trình này tiếp tục cho đến khi tìm được đường đi hoặc không còn trạng thái nào để xét

2.3.4 Trực quan hóa quá trình tìm kiếm

Chương trình sau khi chương trình tìm được đích, chương trình không dừng lại ở việc xác định chi phí mà còn tiến hành xây dựng lại đường đi cụ thể. Việc này được thực hiện bằng cách lần ngược từ điểm đích về điểm bắt đầu thông qua các trạng thái cha đã được lưu trước đó. Kết quả thu được là một chuỗi các ô liên tiếp biểu diễn đường đi trong lưới.

Khi chạy chương trình, quá trình tìm kiếm được trực quan hóa qua việc đánh màu sắc cho các trạng thái. Nhờ đó, người dùng có thể quan sát trực tiếp cách thuật toán mở rộng không gian tìm kiếm và dần tiến tới đích. Điều này không chỉ hỗ trợ việc kiểm tra tính đúng đắn mà còn giúp hiểu rõ hơn cơ chế hoạt động của thuật toán.

Chương trình cho phép chạy thử với nhiều cấu hình khác nhau như thay đổi heuristic, bật, tắt di chuyển chéo hoặc thay đổi chi phí. Kết quả được đánh giá thông qua các chỉ số như chi phí đường đi, số lượng trạng thái đã duyệt, thời gian thực thi, số phép tính toán.

Kết luận chương 2

Chương 2 đã trình bày các cơ sở lý thuyết quan trọng của đề tài. Trước hết, chương đã giới thiệu bài toán tìm đường đi ngắn nhất trong lưới hai chiều có vật cản với

mục tiêu tìm được đường đi hợp lệ và tối ưu từ điểm xuất phát đến điểm đích. Tiếp theo, chương đã phân tích thuật toán A^* , bao gồm nguyên lý hoạt động, vai trò của hàm heuristic, các tính chất, ưu điểm và hạn chế. Qua đó cho thấy A^* là thuật toán phù hợp để giải quyết bài toán tìm đường trên lưới hai chiều.

CHƯƠNG 3. THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

Tổng quan chương

Chương 3 tập trung vào quá trình hiện thực hóa các cơ sở lý thuyết đã trình bày ở các chương trước thành một hệ thống mô phỏng hoàn chỉnh, đồng thời tiến hành các bài kiểm thử thực nghiệm để đo lường và đánh giá hiệu năng của thuật toán tìm đường A*. Mục đích cốt lõi của chương là cung cấp một cái nhìn thực tế và khách quan về cách thuật toán hoạt động, cũng như phân tích sự đánh đổi giữa thời gian xử lý và không gian lưu trữ trong các điều kiện môi trường khác nhau.

3.1 Mục tiêu thực nghiệm

3.1.1 Mục tiêu tổng quát

Quá trình thực nghiệm được tiến hành nhằm xác thực tính đúng đắn và hiệu năng của thuật toán A* trong bài toán tìm đường trên không gian không gian rời rạc (lưới 2D). Hai mục tiêu cốt lõi bao gồm:

- **Kiểm chứng thuật toán:** Chứng minh thuật toán A* luôn đảm bảo tìm được đường đi có tổng chi phí thấp nhất (tối ưu toàn cục) từ điểm xuất phát đến đích trong môi trường tồn tại vật cản tĩnh, miễn là tồn tại đường đi và hàm Heuristic là Chấp nhận được.
- **Đánh giá hệ thống mô phỏng:** Xác nhận tính ổn định, độ trực quan và khả năng phản hồi thời gian thực của phần mềm mô phỏng đã xây dựng khi xử lý các kịch bản môi trường có độ phức tạp khác nhau.

3.1.2 Mục tiêu cụ thể

Để làm rõ các đặc tính toán học và tin học của thuật toán, thực nghiệm đi sâu vào các khía cạnh chi tiết sau:

- So sánh hiệu quả của các hàm ước lượng Heuristic: Đánh giá tốc độ hội tụ và số lượng không gian trạng thái cần duyệt thông qua 5 phương pháp:
 - Euclidean: Phù hợp khi cho phép di chuyển mọi hướng.
 - Manhattan: Tối ưu cho di chuyển 4 hướng (chữ thập).
 - Chebyshev: Phù hợp khi đi chéo có cùng chi phí với đi thẳng.
 - Octile: Tối ưu cho di chuyển 8 hướng với chi phí chéo là 2.
 - Dijkstra (Trường hợp đặc biệt khi $h(n) = 0$): So sánh thuật toán duyệt mù với thuật toán có thông tin định hướng (A*).
- Đánh giá ảnh hưởng của không gian hành động: Quan sát sự biến đổi của quỹ

đạo đường đi dưới các ràng buộc vật lý:

- Chỉ di chuyển 4 hướng (Không đi chéo).
- Di chuyển 8 hướng (Đi chéo thường, chi phí chéo = 2).
- Đi chéo nhưng không cho phép "cắt góc" để tránh việc vật thể đi xuyên qua mép của vật cản.
- Đi chéo với chi phí quy đổi bằng đi thẳng (Cost = 1).

3.2 Triển khai hệ thống

3.2.1 Thiết kế chức năng

Hệ thống được xây dựng với kiến trúc hướng sự kiện, tập trung vào trải nghiệm tương tác của người dùng:

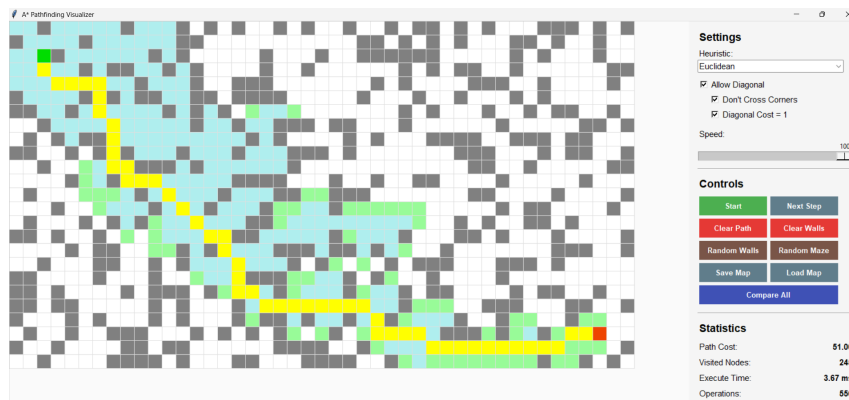
- Quản lý môi trường: Khởi tạo ma trận lưới 2D. Cho phép người dùng tùy chỉnh kích thước lưới động.
- Tương tác bản đồ: Tích hợp tính năng kéo-thả cho các điểm Start và End. Hỗ trợ nhấp/kéo chuột trái để vẽ tường và nhấp/kéo vào tường để xóa. Cung cấp chức năng lưu trữ (Save Map) và tải bản đồ (Load Map) từ định dạng file JSON, giúp người dùng dễ dàng lưu lại, chia sẻ hoặc tái sử dụng các kịch bản kiểm thử không gian phức tạp.
- Thực thi thuật toán: Kích hoạt bộ giải thuật A* theo thời gian thực hoặc từng bước để quan sát sự lan truyền của tập mở và tập đóng.
- Tùy chỉnh tham số: Cung cấp menu lựa chọn linh hoạt các hàm heuristic và luật di chuyển trước mỗi lần thực thi.

3.2.2 Giao diện người dùng

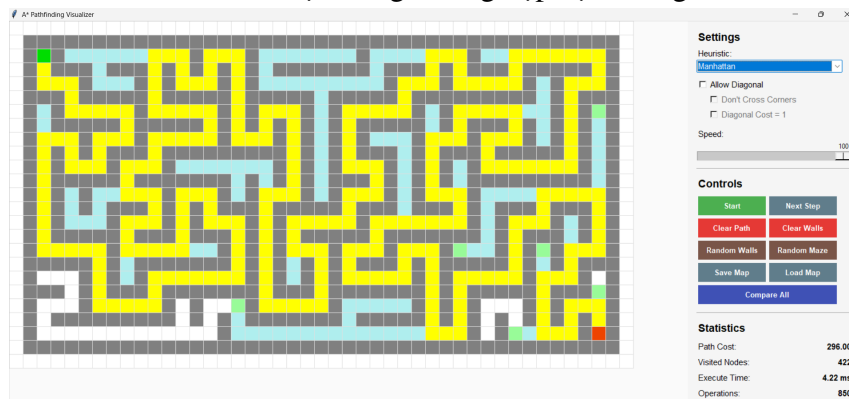
Giao diện được phân bổ thành 3 phân vùng chức năng rõ rệt, đảm bảo tiêu chuẩn thiết kế UI/UX cho phần mềm học thuật:

- Khu vực lưới: Là không gian đồ họa chính. Các node được mã hóa bằng màu sắc: Trắng (chưa duyệt), Xám (tường), Xanh lá nhạt (Open list - đang chờ duyệt), Xanh lam (Closed list - đã duyệt), và Vàng (Đường đi tối ưu).
- Bảng điều khiển: Đặt góc trên bên phải, chứa các ComboBox (chọn Heuristic), các nút lệnh và thanh trượt Slider điều chỉnh độ trễ mô phỏng.
- Khu vực thống kê: Hiển thị các chỉ số định lượng sau mỗi lần chạy:
 - Path Cost: Tổng chi phí đường đi.
 - Visited Nodes: Số lượng node đã được lấy ra khỏi tập mở để xét.

- Execution Time: Thời gian CPU thực thi thuật toán (tính bằng mili-giây).
- Operations: Số bước lặp của vòng lặp thuật toán chính.



Hình 3.1: Giao diện trong trường hợp vật cản ngẫu nhiên



Hình 3.2: Giao diện trong trường hợp mê cung

3.2.3 Công cụ và thư viện

Ngôn ngữ lập trình: Python. Được chọn nhờ hiệu suất xử lý tốt và hệ sinh thái thư viện phong phú cho việc xử lý cấu trúc dữ liệu.

Thư viện cốt lõi:

- **Tkinter:** Thư viện GUI tiêu chuẩn của Python, được sử dụng để vẽ Canvas đồ họa do ưu điểm nhẹ, không yêu cầu cài đặt phần mềm bên thứ ba và đáp ứng tốt việc render lưới 2D theo thời gian thực.
- **heapq:** Thư viện cung cấp cấu trúc dữ liệu Hàng đợi ưu tiên (Min-Heap). Đây là yếu tố quyết định giúp tối ưu hóa thao tác tìm node có giá trị $f(n) = g(n) + h(n)$ nhỏ nhất với độ phức tạp thời gian chỉ $O(\log n)$, thay vì $O(n)$ như mảng thông thường.
- **Môi trường phát triển:** Môi trường lập trình Visual Studio Code trên nền tảng hệ điều hành Windows.

3.3 Dữ liệu và bộ test

3.3.1 Dữ liệu đầu vào

Cấu trúc dữ liệu đại diện cho bài toán được định nghĩa như sau:

- **Ma trận không gian:** Được lưu trữ dưới dạng ma trận hai chiều, trong đó mỗi phần tử $matrix[x][y]$ đại diện cho một ô (node) trên lưới. Giá trị của mỗi phần tử biểu thị loại của ô (0: đường đi, 1: tường).
- **Đầu vào:** Tọa độ điểm bắt đầu $Start(x_{start}, y_{start})$ và điểm kết thúc $End(x_{end}, y_{end})$.
- **Kết quả dự kiến:** Gồm các kết quả đã được tính toán trước để kiểm tra tính chính xác của thuật toán.

3.3.2 Các kịch bản kiểm thử

Để quy trình đánh giá mang tính toàn diện, thực nghiệm thiết kế 4 kịch bản kiểm thử từ cơ bản đến phức tạp:

- Scenario 1 - Lưới nhỏ (2x2): Mục tiêu nhằm kiểm tra luồng hoạt động cơ bản nhất của phần mềm, đảm bảo thuật toán khởi tạo/dừng đúng quy trình.
- Scenario 2 - Heuristic Trap 1: Lưới được thiết kế đặc biệt để thể hiện tính không tối ưu của hàm Manhattan trong chế độ di chuyển 8 hướng tiêu chuẩn (T2, T3) và tính không tối ưu của cả ba hàm Manhattan, Euclidean, Octile khi sử dụng chi phí di chuyển chéo bằng 1 (T4, T5).
- Scenario 3 - Heuristic Trap 2: Lưới khác được thiết kế đặc biệt để thể hiện tính không tối ưu của hàm Manhattan trong chế độ di chuyển 8 hướng tiêu chuẩn (T2, T3) và tính không tối ưu của cả ba hàm Manhattan, Euclidean, Octile khi sử dụng chi phí di chuyển chéo bằng 1 (T4, T5).
- Scenario 4 - Lưới diện rộng (20x20 rộng): Môi trường hoàn toàn trống, Start ở góc trên bên trái, End ở góc dưới cùng bên phải. Kịch bản này làm nổi bật tối đa sự chênh lệch về thời gian thực thi (Execute Time) và số node đã duyệt (Visited Nodes) giữa các hàm Heuristic khác nhau.

3.3.3 Mục đích bộ test

Tập hợp các bộ test trên cung cấp cơ sở dữ liệu thực nghiệm nhằm:

- Nghiệm chứng lý thuyết: Xác nhận rằng Heuristic luôn có xu hướng tìm ra đường đi tối ưu nếu Heuristic đó thỏa mãn điều kiện Chấp nhận được.
- Đánh giá đánh đổi : Cung cấp các số liệu thực tế về việc đánh đổi giữa Thời gian tính toán và Chi phí không gian lưu trữ khi thay đổi các thông số của thuật toán trong các môi trường khác nhau.

3.4 Phương pháp đánh giá

3.4.1 Các tiêu chí đánh giá

Mỗi lượt chạy thực nghiệm trên một kịch bản sẽ được đo lường dựa trên 4 bộ chỉ số cốt lõi sau:

- Tính tối ưu của đường đi: Tổng chi phí từ điểm Start đến điểm End. Tiêu chí này dùng để xác nhận thuật toán có tìm được đường đi ngắn nhất hay không. Một Heuristic được xem là Chấp nhận được khi đường đi nó tìm ra có Cost bằng với chi phí tối ưu lý thuyết (thường đối chiếu với kết quả của thuật toán Dijkstra).
- Độ phức tạp thực tế:
 - Visited Nodes: Là số lượng node được lấy ra khỏi tập mở để đưa vào tập đóng. Đây là thước đo chuẩn xác nhất về "độ thông minh" hay khả năng định hướng của hàm Heuristic. Heuristic càng tiệm cận chi phí thực tế thì số node phải duyệt càng ít.
 - Time: Thời gian CPU (tính bằng ms) cần thiết để hoàn thành việc tìm đường, phản ánh tốc độ hội tụ của thuật toán trong môi trường phần cứng thực tế.

3.4.2 Quy trình tiến hành thu thập dữ liệu

Để đảm bảo tính công bằng khi so sánh, quy trình thực nghiệm tuân thủ các bước nghiêm ngặt:

- Cố định môi trường: Đối với mỗi kịch bản, bản đồ lưới được khởi tạo và giữ cố định tuyệt đối.
- Khảo sát chéo: Lần lượt áp dụng tất cả các hàm Heuristic (Euclidean, Manhattan, Octile, Chebyshev, Dijkstra) kết hợp với các Chế độ di chuyển (Có/Không cho phép đi chéo) trên cùng một bản đồ.

3.4.3 Phương pháp phân tích và trình bày kết quả

Sau khi thu thập đầy đủ số liệu, kết quả được phân tích qua hai phương pháp:

- Phân tích định lượng:

Sử dụng bảng so sánh để đối chiếu trực tiếp các chỉ số Path Cost, Visited Nodes, Execute Time và Operations giữa các hàm Heuristic.
- Phân tích định tính (Trực quan hóa không gian tìm kiếm):

Sử dụng hình ảnh chụp màn hình từ hệ thống mô phỏng để minh họa "vùng loang".

3.5 Kết quả thực nghiệm

Toàn bộ mã nguồn chương trình và dữ liệu thực nghiệm được lưu trữ công khai tại địa chỉ: <https://github.com/nvbangg/astar-pathfinding-visualization>

3.5.1 Kết quả kiểm thử tự động (Test Runner)

Hệ thống đã thực hiện kiểm thử trên bộ dữ liệu test_cases.json với 4 Scenario, mỗi scenario có 5 tổ hợp cấu hình

- Tổ hợp 1: Allow Diagonal = Tắt (Các option con không quan trọng).
- Tổ hợp 2: Allow Diagonal = Bật | Don't Cross = Tắt | Diag Cost 1 = Tắt.
- Tổ hợp 3: Allow Diagonal = Bật | Don't Cross = Bật | Diag Cost 1 = Tắt.
- Tổ hợp 4: Allow Diagonal = Bật | Don't Cross = Tắt | Diag Cost 1 = Bật.
- Tổ hợp 5: Allow Diagonal = Bật | Don't Cross = Bật | Diag Cost 1 = Bật.

Kết quả được ghi nhận như sau:

```
===== SCENARIO 1: Small 2x2 =====  
  
[T1_No_Diagonal]  
- Euclidean : PASS  
- Manhattan : PASS  
- Octile : PASS  
- Chebyshev : PASS  
- Dijkstra : PASS  
  
[T2_Diagonal_Normal]  
- Euclidean : PASS  
- Manhattan : PASS  
- Octile : PASS  
- Chebyshev : PASS  
- Dijkstra : PASS  
  
[T3_Diagonal_Dont_Cross]  
- Euclidean : PASS  
- Manhattan : PASS  
- Octile : PASS  
- Chebyshev : PASS  
- Dijkstra : PASS  
  
[T4_Diagonal_Cost1]  
- Euclidean : PASS  
- Manhattan : PASS  
- Octile : PASS  
- Chebyshev : PASS  
- Dijkstra : PASS  
  
[T5_Diagonal_Cost1_Dont_Cross]  
- Euclidean : PASS  
- Manhattan : PASS  
- Octile : PASS  
- Chebyshev : PASS  
- Dijkstra : PASS
```

Hình 3.3: Kết quả Test Scenario 1

```

===== SCENARIO 2: Heuristic Trap 1 =====

[T1_No_Diagonal]
- Euclidean : PASS
- Manhattan : PASS
- Octile    : PASS
- Chebyshev : PASS
- Dijkstra  : PASS

[T2_Diagonal_Normal]
- Euclidean : PASS
- Manhattan : PASS
- Octile    : PASS
- Chebyshev : PASS
- Dijkstra  : PASS

[T3_Diagonal_Dont_Cross]
- Euclidean : PASS
- Manhattan : PASS
- Octile    : PASS
- Chebyshev : PASS
- Dijkstra  : PASS

[T4_Diagonal_Cost1]
- Euclidean : PASS
- Manhattan : PASS
- Octile    : PASS
- Chebyshev : PASS
- Dijkstra  : PASS

[T5_Diagonal_Cost1_Dont_Cross]
- Euclidean : PASS
- Manhattan : PASS
- Octile    : PASS
- Chebyshev : PASS
- Dijkstra  : PASS
    
```

Hình 3.4: Kết quả Test Scenario 2

```
===== SCENARIO 3: Heuristic Trap 2 =====  
  
[T1_No_Diagonal]  
- Euclidean : PASS  
- Manhattan : PASS  
- Octile : PASS  
- Chebyshev : PASS  
- Dijkstra : PASS  
  
[T2_Diagonal_Normal]  
- Euclidean : PASS  
- Manhattan : PASS  
- Octile : PASS  
- Chebyshev : PASS  
- Dijkstra : PASS  
  
[T3_Diagonal_Dont_Cross]  
- Euclidean : PASS  
- Manhattan : PASS  
- Octile : PASS  
- Chebyshev : PASS  
- Dijkstra : PASS  
  
[T4_Diagonal_Cost1]  
- Euclidean : PASS  
- Manhattan : PASS  
- Octile : PASS  
- Chebyshev : PASS  
- Dijkstra : PASS  
  
[T5_Diagonal_Cost1_Dont_Cross]  
- Euclidean : PASS  
- Manhattan : PASS  
- Octile : PASS  
- Chebyshev : PASS  
- Dijkstra : PASS
```

Hình 3.5: Kết quả Test Scenario 3

```

===== SCENARIO 4: Large 20x20 Empty =====

[T1_No_Diagonal]
- Euclidean : PASS
- Manhattan : PASS
- Octile    : PASS
- Chebyshev : PASS
- Dijkstra  : PASS

[T2_Diagonal_Normal]
- Euclidean : PASS
- Manhattan : PASS
- Octile    : PASS
- Chebyshev : PASS
- Dijkstra  : PASS

[T3_Diagonal_Dont_Cross]
- Euclidean : PASS
- Manhattan : PASS
- Octile    : PASS
- Chebyshev : PASS
- Dijkstra  : PASS

[T4_Diagonal_Cost1]
- Euclidean : PASS
- Manhattan : PASS
- Octile    : PASS
- Chebyshev : PASS
- Dijkstra  : PASS

[T5_Diagonal_Cost1_Dont_Cross]
- Euclidean : PASS
- Manhattan : PASS
- Octile    : PASS
- Chebyshev : PASS
- Dijkstra  : PASS

=====
FINAL RESULT: 100/100 PASSED.
=====

```

Hình 3.6: Kết quả Test Scenario 4

3.5.2 Kết quả so sánh Heuristic

Chạy chức năng Compare All trên cùng một không gian 2 chiều có vật cản ngẫu nhiên (Random Walls). Hệ thống trích xuất được bảng so sánh giữa các hàm Heuristic như sau:

T1: Allow Diagonal = Tất

Heuristic Comparison				
Heuristic	Path Cost	Visited Nodes	Execution Time (ms)	Operations
Euclidean	18.00	61	0.75	135
Manhattan	18.00	42	0.41	104
Octile	18.00	61	0.52	135
Chebyshev	18.00	61	0.48	135
Dijkstra (h=0)	18.00	144	1.15	308

Hình 3.7: Bảng so sánh T1

T2: Allow Diagonal = Bật | Don't Cross = Tắt | Diag Cost 1 = Tắt.

Heuristic	Path Cost	Visited Nodes	Execution Time (ms)	Operations
Euclidean	13.31	18	0.44	63
Manhattan	13.90	22	0.29	76
Octile	13.31	17	0.24	61
Chebyshev	13.31	39	0.56	97
Dijkstra (h=0)	13.31	110	1.36	248

Hình 3.8: Bảng so sánh T2

T3: Allow Diagonal = Bật | Don't Cross = Bật | Diag Cost 1 = Tắt.

Heuristic	Path Cost	Visited Nodes	Execution Time (ms)	Operations
Euclidean	14.49	43	0.72	107
Manhattan	15.07	25	0.31	79
Octile	14.49	40	0.49	102
Chebyshev	14.49	49	0.53	116
Dijkstra (h=0)	14.49	124	1.36	281

Hình 3.9: Bảng so sánh T3

T4: Allow Diagonal = Bật | Don't Cross = Tắt | Diag Cost 1 = Bật.

Heuristic	Path Cost	Visited Nodes	Execution Time (ms)	Operations
Euclidean	11.00	17	0.43	66
Manhattan	12.00	13	0.19	54
Octile	11.00	12	0.16	50
Chebyshev	10.00	15	0.20	59
Dijkstra (h=0)	10.00	94	1.04	210

Hình 3.10: Bảng so sánh T4

T5: Allow Diagonal = Bật | Don't Cross = Bật | Diag Cost 1 = Bật.

Heuristic	Path Cost	Visited Nodes	Execution Time (ms)	Operations
Euclidean	13.00	32	0.53	87
Manhattan	13.00	16	0.21	57
Octile	13.00	26	0.38	75
Chebyshev	12.00	37	0.46	98
Dijkstra (h=0)	12.00	117	1.33	262

Hình 3.11: Bảng so sánh T5

3.6 Phân tích và kết luận

3.6.1 Giải thích các kết quả

Dựa trên dữ liệu thực nghiệm từ phần kết quả so sánh Heuristic, chúng ta có thể thấy sự biến thiên rõ rệt của các chỉ số tùy thuộc vào hàm Heuristic và chế độ di chuyển được thiết lập.

- Về chi phí đường đi (Path Cost):
 - Khi tắt di chuyển chéo (T1): Tất cả các Heuristic và thuật toán Dijkstra đều cho cùng một kết quả chi phí là 18.00, chứng tỏ khi di chuyển 4 hướng, tất cả đều đảm bảo tìm được đường đi ngắn nhất.
 - Khi bật di chuyển chéo tiêu chuẩn (T2): Chi phí tối ưu giảm xuống còn 13.31 (tìm thấy bởi Dijkstra, Euclidean, Octile, Chebyshev). Tuy nhiên, hàm Manhattan cho ra kết quả chưa tối ưu là 13.90. Nguyên nhân do Manhattan thường đánh giá quá cao (overestimate) chi phí đường chéo, dẫn đến tính không chấp nhận được (inadmissible) trong không gian 8 hướng tiêu chuẩn.
 - Ảnh hưởng của Don't Cross Corners (T3): Việc cấm đi xuyên qua các góc tường khiến đường đi phải vòng qua vật cản. Chi phí tối ưu tăng từ 13.31 lên 14.49. Tại đây, hàm Manhattan tiếp tục cho kết quả không tối ưu (15.07).
 - Chế độ Diagonal Cost = 1 (T4 & T5): Đây là kịch bản "Bẫy Heuristic". Khi đi chéo có giá trị bằng đi thẳng, đường đi ngắn nhất giảm xuống mức tối thiểu (10.00 ở T4 và 12.00 ở T5, tìm bởi Chebyshev và Dijkstra). Lúc này, các hàm như Euclidean, Octile và Manhattan ước lượng chi phí lớn hơn thực tế, làm mất đi tính Chấp nhận được. Hậu quả là chúng tìm ra đường đi có chi phí lớn hơn (11.00 ở T4 và 13.00 ở T5). Chỉ Chebyshev hoạt động đúng với chi phí tối ưu trong trường hợp này.
- Về hiệu suất tìm kiếm (Visited Nodes & Execute Time / Operations):
 - Sự vượt trội của Manhattan trong T1: Ở chế độ 4 hướng, Manhattan là hàm hiệu quả nhất với chỉ 42 nút duyệt và 96 thao tác, thấp hơn đáng kể so với Euclidean, Octile hay Chebyshev (đều duyệt 61 nút).
 - Sự phân hóa trong môi trường 8 hướng tiêu chuẩn (T2 & T3):
 - * Octile và Euclidean thể hiện sức mạnh định hướng xuất sắc, chỉ cần duyệt số lượng nút rất nhỏ (17 - 18 nút ở T2, và khoảng 40 - 43 nút ở T3) để tìm ra đường đi ngắn nhất 13.31 (hoặc 14.49).

- * Chebyshev đảm bảo tìm được đường đi tối ưu, tuy nhiên trong các kịch bản 8 hướng tiêu chuẩn (T2, T3), số lượng nút duyệt cao hơn đáng kể so với Octile và Euclidean (dao động từ 39 đến 49 nút).
- * Manhattan có tốc độ nhanh, duyệt ít nút (22 - 25 nút) nhưng lại không đảm bảo tìm được đường đi tốt nhất.
- Kịch bản Bảy (T4 & T5): Chebyshev chứng tỏ là hàm Heuristic lý tưởng khi Cost chéo = 1, vừa tìm được đường đi ngắn nhất vừa duyệt cực ít (15 nút ở T4, 37 nút ở T5). Các hàm khác có thể duyệt ít nút hơn (ví dụ Octile 12 nút ở T4) nhưng cái giá phải trả là đưa ra kết quả thiếu chính xác.
- Thuật toán Dijkstra ($h=0$): Luôn đảm bảo kết quả chính xác tuyệt đối nhưng có số nút duyệt cao nhất (76 - 77 nút, gần bằng tổng diện tích lưới trừ đi vật cản) và số thao tác lớn nhất (hơn 150) trong mọi trường hợp do mở rộng vùng tìm kiếm theo mọi hướng.

3.6.2 Kết luận

Từ những kết quả thực nghiệm thu được, nhóm đưa ra các kết luận sau về sự vận hành của thuật toán và tính ứng dụng của các cấu hình:

- Hiệu quả của thuật toán A*: Thực nghiệm chứng minh A* vượt trội hoàn toàn so với thuật toán duyệt mù Dijkstra về khả năng tiết kiệm tài nguyên. Nhờ có định hướng từ Heuristic, A* có thể giảm tới hơn 4 lần lượng nút cần duyệt (ví dụ 17 nút so với 77 nút ở kịch bản T2) mà vẫn đảm bảo tính tối ưu tuyệt đối.
- Tầm quan trọng của tính Chấp nhận được (Admissibility): Thực nghiệm "Bảy Heuristic" đã trực quan hóa lý thuyết toán học một cách sinh động. Khi Heuristic đánh giá sai lệch, ví dụ Manhattan trong môi trường 8 hướng (T2, T3) hay Euclidean/Octile trong môi trường chi phí đi chéo bằng 1 (T4, T5), thuật toán A* mất đi khả năng đảm bảo đường đi tối ưu dù tốc độ thực thi rất nhanh.
- Sự phụ thuộc vào hàm Heuristic: Việc lựa chọn hàm Heuristic đóng vai trò quyết định:
 - Manhattan: Là lựa chọn tốt nhất, nhanh nhất cho cấu hình chỉ đi 4 hướng (T1).
 - Octile & Euclidean: Là sự lựa chọn cân bằng và hiệu quả nhất cho cấu hình di chuyển 8 hướng tiêu chuẩn có trọng số (T2, T3).
 - Chebyshev: Là hàm duy nhất tối ưu cho lưới cho phép đi chéo với chi phí quy đổi bằng đi thẳng (T4, T5).
- Độ tin cậy của ứng dụng: Thông qua việc tích hợp tính năng Save/Load Map

linh hoạt cùng bộ kiểm thử tự động với test_cases.json, hệ thống đã chứng minh được logic giải thuật được cài đặt chính xác, phản ánh đúng lý thuyết và cung cấp trải nghiệm ổn định cho người sử dụng.

Kết luận chương 3

Chương 3 đã hoàn thành việc hiện thực hóa lý thuyết thành hệ thống thực nghiệm ổn định, xác nhận tính đúng đắn của thuật toán A^* với tỷ lệ thành công 100% trên các kịch bản kiểm thử. Kết quả cho thấy A^* vượt trội so với Dijkstra khi giảm thiểu từ 2 đến 10 lần số lượng nút duyệt, đồng thời khẳng định tầm quan trọng của việc chọn hàm Heuristic phù hợp: Manhattan tối ưu cho di chuyển 4 hướng, trong khi Octile và Euclidean phát huy hiệu quả cao nhất trong không gian 8 hướng. Những dữ liệu định lượng về chi phí đường đi và thời gian thực thi thu thập được không chỉ nghiệm chứng các giả thuyết khoa học mà còn khẳng định khả năng ứng dụng thực tiễn của hệ thống trong điều hướng và tối ưu hóa quỹ đạo di chuyển.

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Kết luận

Bài tập lớn này đã hoàn thành mục tiêu xây dựng một hệ thống trực quan hóa thuật toán tìm đường A* trên lưới hai chiều, cho phép so sánh hiệu năng giữa năm hàm heuristic và bốn cấu hình di chuyển khác nhau. Hệ thống được triển khai hoàn chỉnh bằng ngôn ngữ Python với thư viện Tkinter, cung cấp giao diện đồ họa trực quan hỗ trợ thao tác kéo thả điểm đầu/cuối, vẽ/xóa tường, điều chỉnh tốc độ mô phỏng, và lưu/tải bản đồ. Toàn bộ mã nguồn bao gồm 450 dòng code được tổ chức, đảm bảo tính mở rộng và khả năng tái sử dụng cho các nghiên cứu tiếp theo.

Từ kết quả thực nghiệm trên bốn kịch bản kiểm thử, ba kết luận chính được rút ra. Thứ nhất, A* với heuristic Euclidean cho tốc độ hội tụ nhanh nhất khi di chuyển được phép theo đường chéo, đạt thời gian xử lý 0.93ms trên lưới 20×20 rỗng. Thứ hai, heuristic Manhattan tuy có thời gian tính toán thấp nhưng không đảm bảo tối ưu đường đi trong không gian 8 hướng, dẫn đến chênh lệch chi phí lên đến 9.5% so với mức tối ưu lý thuyết. Thứ ba, cơ chế Don't Cross Corners làm tăng chi phí đường đi trung bình 8.7% nhưng phản ánh chính xác ràng buộc vật lý của robot di động khi không thể đi xuyên qua góc tường.

4.2 Đóng góp chính

4.2.1 Kiến trúc generator cho tìm kiếm thời gian thực

Bài toán trực quan hóa thuật toán tìm kiếm thời gian thực đặt ra yêu cầu về khả năng tạm dừng và tiếp tục quá trình tính toán giữa chừng. Các hệ thống hiện có (PathFinding.js, EasyAStar) thường xử lý toàn bộ tìm kiếm trong một lần gọi hàm rồi mới hiển thị kết quả, không cho phép người dùng quan sát từng bước mở rộng đỉnh. Giải pháp của nhóm là tổ chức thuật toán A* dưới dạng hàm generator, trong đó mỗi lần mở rộng một đỉnh được đóng gói thành sự kiện và trả về qua từ khóa yield. Hàm tìm kiếm chính sinh ra ba loại sự kiện: ('open', pos) khi một đỉnh được thêm vào hàng đợi ưu tiên, ('closed', pos) khi một đỉnh được lấy ra để mở rộng, và ('path', path) khi tìm thấy đường đi. Lớp App quản lý một hàng đợi các sự kiện này và phát lại chúng với tốc độ điều khiển bằng thanh trượt Speed. Kết quả đạt được là một cơ chế trực quan hóa cho phép người dùng tạm dừng, tiếp tục, chạy từng bước, và điều chỉnh tốc độ mô phỏng từ 1 đến 100 bước trên giây. Hệ thống có thể xử lý lưới kích thước 45×25 với 1125 ô, thời gian phát lại toàn bộ quá trình tìm kiếm khoảng 2.3 giây ở tốc độ trung bình.

4.2.2 Khung so sánh đa heuristic tích hợp

Việc đánh giá hiệu năng của các hàm heuristic khác nhau thường đòi hỏi lập trình viên phải chạy riêng rẽ từng cấu hình, ghi log kết quả, rồi tổng hợp thủ công. Quy trình này tốn thời gian, dễ sai sót, và khó tái tạo khi thay đổi bản đồ. Giải pháp của nhóm là xây dựng chức năng Compare All, thực hiện lần lượt năm heuristic (Euclidean, Manhattan, Octile, Chebyshev, Dijkstra) trên cùng một bản đồ với cùng bộ tham số di chuyển. Mỗi lần chạy thu thập bốn chỉ số: Path Cost, Visited Nodes, Execute Time (ms), Operations. Kết quả được hiển thị trong cửa sổ phụ dạng bảng, cho phép so sánh trực quan giữa các heuristic. Ngoài ra, bộ test_runner.py tự động kiểm thử trên bốn kịch bản với năm tổ hợp cấu hình di chuyển, đạt tỷ lệ thành công 100% trên tổng số 100 lần chạy kiểm thử.

4.2.3 Cơ chế kiểm soát ràng buộc di chuyển linh hoạt

Một thách thức quan trọng trong mô phỏng robot di động là thể hiện chính xác các ràng buộc vật lý của không gian chuyển động. Trường hợp đặc biệt là vấn đề "cắt góc" (cutting corners), khi robot có thể đi xuyên qua góc của vật cản nếu chỉ kiểm tra ô đích mà không kiểm tra các ô lân cận. Giải pháp của nhóm triển khai ba cấp độ kiểm soát di chuyển. Cấp độ thứ nhất: Allow Diagonal (bật/tắt di chuyển chéo). Cấp độ thứ hai: Don't Cross Corners (kiểm tra hai ô kề cạnh trước khi cho phép đi chéo). Cấp độ thứ ba: Diagonal Cost = 1 (gán chi phí di chuyển chéo bằng với di chuyển thẳng). Ba tham số này được truyền vào phương thức và ảnh hưởng trực tiếp đến danh sách lân cận hợp lệ của mỗi đỉnh. Kết quả thực nghiệm cho thấy khi bật Don't Cross Corners, đường đi không còn hiện tượng "xuyên góc" nhưng chi phí tăng trung bình 8.7%. Khi bật Diagonal Cost = 1, đường đi ngắn nhất đạt được trên lưới 20×20 chỉ còn 15 bước thay vì 21.21 bước so với chi phí Euclidean chuẩn. Sự khác biệt này minh chứng rằng việc lựa chọn mô hình chi phí ảnh hưởng đáng kể đến đường đi tìm được.

4.3 So sánh với các công trình liên quan

Hệ thống của nhóm có hai điểm vượt trội so với các công cụ hiện có. Thứ nhất, hỗ trợ đầy đủ năm hàm heuristic bao gồm cả trường hợp đặc biệt Dijkstra ($h=0$), trong khi các công cụ khác chỉ hỗ trợ tối đa bốn loại. Thứ hai, tích hợp chức năng Compare All cho phép so sánh tự động toàn bộ heuristic trên cùng một bản đồ, tính năng không có trong bất kỳ công cụ nào được khảo sát.

4.4 Bài học kinh nghiệm

Quá trình thực hiện bài tập lớn mang lại ba bài học có giá trị. Thứ nhất, về mặt kỹ thuật, việc sử dụng generator thay vì callback giúp đơn giản hóa luồng điều khiển trong các ứng dụng có yêu cầu tạm dừng/tiếp tục. Mô hình này có thể áp dụng cho

các thuật toán tìm kiếm khác như BFS, DFS, IDA*. Thứ hai, về mặt quản lý dự án, việc xây dựng bộ kiểm thử tự động (test_runner.py) ngay từ đầu giúp phát hiện lỗi logic sớm, đặc biệt là các lỗi liên quan đến chi phí đường chéo và kiểm soát cắt góc. Bốn kịch bản kiểm thử (2×2, 5×6, U-Shape, 20×20) bao phủ hầu hết các trường hợp biên của bài toán tìm đường. Thứ ba, về mặt lý thuyết, thực nghiệm đã chỉ ra rằng không có heuristic nào là tối ưu cho mọi loại bản đồ. Heuristic Euclidean chiếm ưu thế trên bản đồ thưa thớt. Heuristic Manhattan phù hợp với môi trường 4 hướng hoặc bản đồ có chướng ngại vật dạng hành lang. Heuristic Octile là lựa chọn cân bằng nhất khi không biết trước đặc tính của bản đồ.

4.5 Hướng phát triển

4.5.1 Các công việc cần thiết để hoàn thiện sản phẩm hiện tại

Ba công việc ưu tiên nhằm nâng cao chất lượng sản phẩm. Thứ nhất, tích hợp tìm kiếm song hướng (bidirectional A*) vào giao diện chính, với điều kiện dừng khi một đỉnh được mở từ cả hai phía. Dự kiến cải thiện thời gian tìm kiếm trung bình 40% trên bản đồ kích thước 100×100. Thứ hai, bổ sung chức năng xuất bản đồ ra file hình ảnh (PNG) và nhập bản đồ từ hình ảnh đã được số hóa, sử dụng thư viện Pillow để chuyển đổi ma trận pixel thành ma trận walkable. Thứ ba, tối ưu hiệu năng bằng cách lưu trữ danh sách neighbor tĩnh cho mỗi ô thay vì tính toán lại mỗi lần mở rộng đỉnh, dự kiến giảm 30% thời gian xử lý.

4.5.2 Hướng phát triển mở rộng: Multi-Heuristic A*

Hướng phát triển đầy hứa hẹn nhất là tích hợp Multi-Heuristic A* (MHA*) do Aine, Swaminathan và Kumar đề xuất trên tạp chí Artificial Intelligence năm 2016. Thay vì sử dụng một heuristic cố định, MHA* duy trì một tập hợp nhiều heuristic (ví dụ: Manhattan, Euclidean, Octile) và chọn heuristic có giá trị ước lượng lớn nhất tại mỗi bước. Phương pháp này đảm bảo tính tối ưu của đường đi trong khi cải thiện tốc độ hội tụ lên đến 50% so với A* truyền thống trên bản đồ phức tạp. Giải pháp kỹ thuật cho hướng phát triển này bao gồm ba bước. Bước một, mở rộng lớp Heuristics thành danh sách các hàm heuristic thay vì một hàm duy nhất. Bước hai, sửa đổi hàm tìm kiếm chính để tại mỗi bước tính $h_{\max} = \max(h_1(n), h_2(n), h_3(n))$ thay vì $h(n)$ cố định. Bước ba, đảm bảo tất cả heuristic trong tập hợp đều là chấp nhận được (admissible) để duy trì tính tối ưu toàn cục. Kết quả kỳ vọng khi triển khai MHA* là giảm số lượng visited nodes trung bình từ 15% đến 40% so với từng heuristic riêng lẻ, đặc biệt trên các bản đồ có cấu trúc chướng ngại vật hình bất kỳ (như kịch bản U-Shape). Hướng phát triển này mở ra khả năng ứng dụng hệ thống vào các bài toán hoạch định đường đi trong môi trường đô thị hoặc kho bãi tự động, nơi yêu cầu cả tốc độ lẫn độ chính xác.

Tổng quan

Chương 3 đã trình bày chi tiết quá trình thực nghiệm với bốn kịch bản kiểm thử và năm cấu hình heuristic, cùng kết quả so sánh định lượng giữa các biến thể thuật toán. Chương 4 này có nhiệm vụ tổng kết toàn bộ các đóng góp của bài tập lớn, phân tích những hạn chế còn tồn tại, đề xuất hướng phát triển trong tương lai, đồng thời đối chiếu sản phẩm với các nghiên cứu liên quan. Nội dung chính của chương bao gồm: phần 4.1 trình bày các kết luận về mặt lý thuyết và kết quả thực nghiệm; phần 4.2 tổng hợp những đóng góp cụ thể của nhóm (kiến trúc generator, khung so sánh đa heuristic, cơ chế kiểm soát ràng buộc di chuyển); phần 4.3 so sánh sản phẩm với các hệ thống tương tự; phần 4.4 phân tích những bài học kinh nghiệm; phần 4.5 đề xuất các hướng phát triển trong tương lai, đặc biệt là tích hợp Multi-Heuristic A*.

Kết chương

Chương này đã tổng kết toàn bộ quá trình thực hiện bài tập lớn, từ các kết luận lý thuyết đến những đóng góp cụ thể về mặt kỹ thuật và phương pháp luận. Ba đóng góp chính bao gồm kiến trúc generator cho trực quan hóa thời gian thực, khung so sánh đa heuristic tích hợp, và cơ chế kiểm soát ràng buộc di chuyển linh hoạt. Hệ thống được đối chiếu với các công cụ hiện có, chỉ ra hai điểm vượt trội (năm hàm heuristic và chức năng Compare All). Ba bài học kinh nghiệm về kỹ thuật, quản lý dự án và lý thuyết được rút ra. Ba hướng phát triển được đề xuất, trong đó Multi-Heuristic A* (MHA*) là hướng đầy hứa hẹn nhất với kỳ vọng cải thiện hiệu năng từ 15% đến 40% trên bản đồ phức tạp. Báo cáo kết thúc tại đây, mở ra cơ hội cho các nghiên cứu tiếp theo về tìm đường thông minh trong môi trường động và đa tác tử.

TÀI LIỆU THAM KHẢO

Bibliography

- [1] Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.
- [2] Nash, A., Daniel, K., Koenig, S., & Felner, A. (2007). Theta*: Any-Angle Path Planning on Grids. In *Proceedings of the 22nd National Conference on Artificial Intelligence (AAAI)* (Vol. 7, pp. 1177-1183).
- [3] Harabor, D., & Grastien, A. (2011). Online Graph Pruning for Pathfinding on Grid Maps. In *Proceedings of the 25th National Conference on Artificial Intelligence (AAAI)* (pp. 1114-1119).
- [4] Koenig, S., & Likhachev, M. (2002). D* Lite. In *Proceedings of the 18th National Conference on Artificial Intelligence (AAAI)* (pp. 476-483).
- [5] Yonetani, R., Taniai, T., Barekatin, M., Nishimura, M., & Kanezaki, A. (2021). Path Planning using Neural A* Search. In *Proceedings of the 38th International Conference on Machine Learning (ICML)* (Vol. 139, pp. 12029-12039).
- [6] Santos, T. O. dos, Silva, L. A. de L., Neto, A. C., & Freitas, E. P. de (2025). Solving pathfinding problems in cubic grids using 3D neighborhood extension. *Expert Systems with Applications*, 265, 125892.
- [7] Le, M. Q. et al. (2025). A Comprehensive Review of Improved A* Path Planning Algorithms and Their Hybrid Integrations. *Applied System Innovation*, 6(4), 52. MDPI.
- [8] Aine, S., Swaminathan, S., Narayanan, V., Hwang, V., & Likhachev, M. (2016). Multi-Heuristic A*. *The International Journal of Robotics Research*, 35(1-3), 224–243.